

# Массовая отметка записей

В классе ListIterator (далее по тексту "итератор") реализован оптимальный алгоритм для получения выборки в иерархическом и не иерархическом реестре. Главным образом, класс используют для работы с функционалом [массовой отметки](#) записей.

## API класса ListIterator

---

### Подключение

Для использования в C++ импортируйте библиотеку sbis-bl-core/helpers/list\_iterator.hpp.

```
# include <sbis-bl-core/helpers/list_iterator.hpp>
```

Для использования в Python импортируйте модуль [sbis](#). Этот модуль входит в состав модуля бизнес-логики BL Core-Py.

```
from sbis import ListIterator
```

Библиотека и модуль включены в состав [SBIS SDK](#).



#### Примечание

Модуль sbis уже импортирован в [справочниках объектов](#). Следовательно, при разработке кода для метода бизнес-логики, например, [Manually implemented list](#), импортировать модуль sbis не нужно.

### Инициализация

Параметры для инициализации класса доступны по следующим ссылкам:

- [C++](#)
- [Python](#)

### Режим массового чтения

По умолчанию итератор работает в режиме единичного чтения, т.е. отмеченные записи запрашиваются у списочного метода по одной. Для эффективной работы итератора рекомендуется перевести его в режим массового чтения. Для этого вызовите метод **SetMassReading()**.

В режиме массового чтения итератор вызывает списочный метод с аргументом "Фильтр", в который добавляет параметр фильтрации "СписокИдО". Это набор идентификаторов записей, которые должен вернуть списочный метод.

Декларативный списочный метод по умолчанию поддерживает обработку параметра фильтрации "СписокИдО". При использовании метода этого типа дополнительных настроек не потребуется.

Метод, реализуемый вручную, по умолчанию не поддерживает параметр фильтрации "СписокИдО". Следовательно, для работы итератора нужно внести правки в исходный код метода, чтобы поддержать его обработку.

Режим единичного чтения записей итератора оставлен для обратной совместимости с устаревшим кодом. В этом режиме итератор вызывает списочный метод с аргументом "Фильтр", в который добавляет параметр фильтрации "ИдО". Это идентификатор записи, которую должен вернуть списочный метод.

### Отметка с учетом иерархии записей

Для отметки записей с учетом их иерархии итератору нужно явно указать имя поля иерархии (при необходимости, имя поля с признаком узла иерархии). Для этого используйте метод **SetHierarchy()**.

# Изменение поля, где хранятся идентификаторы записей

По умолчанию считается, что идентификатор хранится в первом поле записи. В некоторых сценариях это может быть другое поле.

Вы можете явно указать итератору, в каком из полей записи хранится идентификатор. Для этого используйте метод `SetKeyName()`.

## Навигация

По умолчанию итератор использует постраничную навигацию для выборки записей. Если списочный метод (см. [Инициализация](#)) поддерживает [навигацию по курсору](#), то рекомендуется перевести итератор на механизм курсора. Для этого используйте метод `SetNavigationType()`.



### Автодобавление фильтра `__WithIterativenessInMind`

Фильтр вызываемого списочного метода дополняется полем `"__WithIterativenessInMind"` - маркер об "итеративной природе" вызова.

## Управление обходом списка

Для более точного управления обходом можно применить расширенное API на основе следующих объектов:

- **IterationCallback** — объект, который создается прикладным разработчиком и передается в метод `Iterate`.
- **IterationExecutor** — объект, который создается в `ListIterator` и передается в `Callback` в процессе обхода списка.

`IterationCallback` потребляет данные и может управлять обходом через `IterationExecutor`.

`IterationExecutor` дает информацию о внутренних особенностях процесса итерирования и позволяет влиять на дальнейшие действия итератора.

Сигнатуры метода `Iterate` можно найти в документации для [C++](#) и [Python](#).

Также в составе `IterationExecutor` есть метод **InitialSelection**, который позволяет получить список выбранных записей (отмеченных в реестре). Данный метод возвращает объект класса **SelectionInfo**.

## Пример использования

В качестве примера разберём использование итератора массовой отметки записей реестра. Настройка этого функционала на клиентской части рассмотрена [здесь](#).

Допустим, что изначально в реестре используется декларативный списочный метод `BLO.List`. Для функционала массовой отметки записей нужен новый метод бизнес-логики с итератором, например, `BLO.ListWithIterator`.



### Примечание

Когда метод `BLO.List` [реализуется вручную](#), то не нужно создавать отдельный метод бизнес-логики. Логика обработки параметра фильтрации [selection](#) можно реализовать в коде этого метода.

Разберём как создать метод `BLO.ListWithIterator`.

1. Откройте [Genie](#). Подробнее о Genie вы можете прочитать [здесь](#).
2. Откройте проект вашего приложения.
3. Найдите [справочник объектов](#), где создан объект бизнес-логики "BLO".
4. В этом объекте создайте новый метод с типом [Manually implemented list](#).
5. Для этого метода установите следующие параметры:
  - Name: `ListWithIterator`
  - Accessibility: `Internal Application Contract`
  - Filter parameters:
    - Name: `СписокИдО`
    - Type: `String array`
  - Implementation
    - Method type: `Python/C++`

- o Implementation kind: Code/File

Реализация кода метода должна быть примерно следующей:

```
1 // C++
2 #include <sbis-bl-core/helpers/list_iterator.hpp>
3
4 RPC_FUNC_4( L"BLO.ListWithIterator", RecordSet, ListWithIterator,
5     dbi::StringList, L"ДопПоля", ext_fields,
6     Record, L"Фильтр", filter,
7     dbi::SortingList, L"Сортировка", sort_list,
8     dbi::Navigation, L"Навигация", navigation )
9 {
10     if( !filter.TestField( L"selection"_sv ) )
11     {
12         result = blcore::Object( L"BLO" ).Invoke<RecordSet>( L"List",
13             ext_fields, filter, sort_list, navigation );
14         return;
15     }
16     bl::ListIterator it( L"BLO.List", filter );
17     it.SetHierarchy( L"Раздел" );
18     it.SetMassReading( true );
19     result = RecordSet();
20     it.Iterate( [&]( RecordSet const& rs ) {
21         if( result.Size() == 0 )
22             result = RecordSet( rs.Format() );
23         for( Record const& rec : rs )
24         {
25             result.AddRow( rec );
26         }
27     });
28 }
```

```
1 # Python
2 @staticmethod
3 def ListWithIterator(ДопПоля, Фильтр, Сортировка, Навигация):
4     if Фильтр.TestField("selection") is None:
5         return BLO.List(ДопПоля, Фильтр, Сортировка, Навигация)
6
7     class Result:
8         def __init__(self):
9             self.iterator = sbis.ListIterator("BLO.List", Фильтр)
10             self.iterator.SetHierarchy("Раздел")
11             self.iterator.SetMassReading(True)
12             self.result = None
13
14         def callback(self, rs):
15             if self.result is None:
16                 self.result = sbis.RecordSet(rs.Format())
17             for rec in rs:
18                 self.result.AddRow(rec)
19
20         def get(self):
21             self.iterator.Iterate(self.callback)
22             return self.result
```

## Параметр фильтрации selection

Параметр фильтрации **selection** используется для работы итератора. Это запись ([sbis.Record](#) или [sbis::Record](#)) с полями, которые описаны в следующей таблице.

| Название  | Тип          | Описание                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| marked    | String array | Список(1) идентификаторов записей(2), которые включены в выборку.                                                                                                                                                                                                                                                                                                                                 |
| excluded  | String array | Список(1) идентификаторов записей(2), который исключены из выборки.                                                                                                                                                                                                                                                                                                                               |
| type      | String       | Какие типы записей включать в результирующую выборку.<br>Исключает использование поля types.<br>Возможные значения: <ul style="list-style-type: none"><li>• all — все типы записей(2).</li><li>• leaf — только листья.</li><li>• node — только узлы или скрытые узлы.</li></ul> Если параметр не задан, то в результирующую выборку включаются все типы записей.                                  |
| types     | String array | Какие типы записей включать в результирующую выборку.<br>Массив значений: <ul style="list-style-type: none"><li>• leaf — только листья;</li><li>• node — только узлы;</li><li>• hidden - только скрытые узлы.</li></ul> Если параметр не задан, то в результирующую выборку включаются все типы записей.                                                                                          |
| recursive | Boolean      | Определяет будут ли включен в выборку дочерние листья, если отмечен только узел (скрытый узел), т.е. его идентификатор находится в поле marked.<br>Когда поле установлено в значение false, то в выборку включается только узел (скрытый узел).<br>Когда поле установлено в значение true, то в выборку включается и узел (скрытый узел), и его дочерние листья.<br>Значение по умолчанию — true. |

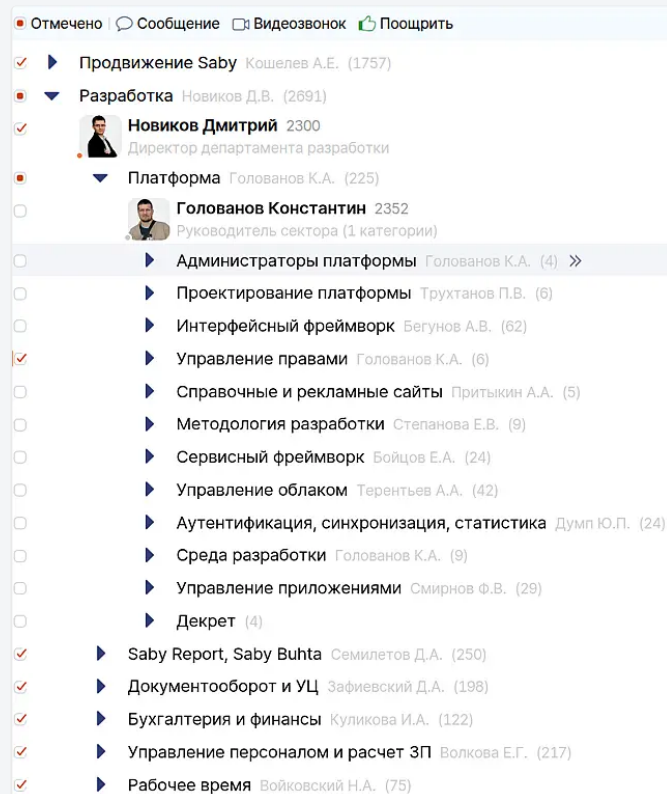
1. Если идентификатор узла (скрытого узла) включен в поля marked и excluded, то считается, что выбрано его содержимое без выбора самого узла (скрытого узла). Если в составе массива marked передано значение NULL, то это означает, что выбрано все (корень выборки). Например, "marked":[null,x,b,c].
2. Здесь говорится о записях всех типов: лист, узел или скрытый узел. Подробнее о каждом типе записей читайте [здесь](#).



### Примечание

Значение NULL передается в составе массива вместе с другими элементами, т.к. возможна ситуация, когда необходимо выбрать некоторый набор записей среди ранее исключенных из выборки.

Например, можно в корневом каталоге исключить какую-нибудь папку, но затем добавить в выборку запись из исключенной папки.



Например, мы в списке отделов компании выбрали все записи. Спустя некоторое время исключили из выборки папку "Платформа". Затем из этой папки выбрали раздел "Управление правами". Соответственно, `marked=[null, "Управление правами"]`, `excluded = ["Платформа"]`.

Параметр фильтрации `selection` добавляется в аргумент "Фильтр" при вызове списочного метода БЛ из клиентской части, когда реестр работает с [массовой отметкой записей](#).

## Алгоритм работы ListIterator

1. Итератор вызывает списочный метод бизнес-логики. При вызове метода итератор добавляет в аргумент "Фильтр" параметр фильтрации "СписокИД" со значением, которое копируется из поля [marked](#). При этом сам `selection` из фильтра удаляется.
2. Итератор получает ответ списочного метода, в котором:
  - исключает записи с идентификаторами [excluded](#).  
В этой точке срабатывает `callback`, который указан при вызове итерации методом `Iterate()`. В этот `callback` передаётся `RecordSet`, который получен итератором.
  - исключает листья — в результате остаются только узлы (скрытые узлы);
3. Итератор вызывает списочный метод бизнес-логики. При вызове метода итератор добавляет в аргумент "Фильтр" параметр фильтрации "Раздел" (по умолчанию). Это имя поля иерархии, которое можно поменять в [настройках](#). В качестве значения такого параметра передаёт массив идентификаторов узлов (скрытых узлов).
4. Итератор получает ответ списочного метода, в котором:
  - исключает записи с идентификаторами `excluded`.  
В этой точке срабатывает `callback`, который указан при вызове итерации методом `Iterate()`. В этот `callback` передаётся `RecordSet`, который получен итератором.
  - исключает листья — в результате остаются только узлы (скрытые узлы);
5. Далее повторяются итерации (п.3 и п.4) до тех пор, пока не будут обработаны все узлы (скрытые узлы). Иными словами, итерации завершаются, когда из ответа исключены листья и полученный набор узлов (скрытых узлов) становится пустым.